

Performance Testing

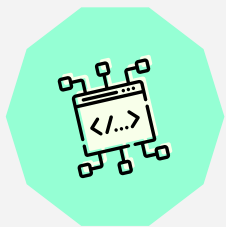
—
Software Testing

01 Performance Testing

02 The Testing Process

03 Testing Scenarios

04 Class Activities



01

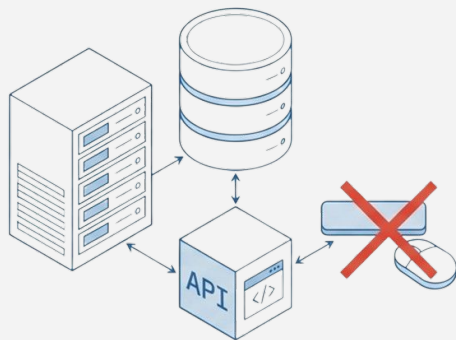
— “Performance Testing”

What: The Anatomy of the Test

Evaluating speed, stability, and scalability under load.



The Subject



Focus on **APIs**,
Database, and **Server**
Capacity.

The Vital Signs



Response Time



Virtual Users (**VUs**)



Rate Error (**HTTP 500**)



Throughput

Testing Methods

Traffic Testing

Load Testing
(Normal traffic)

Stress Testing
(Breaking point)

Spike Testing
(Peak bottleneck)



Volume
Testing



Soak
Testing

Performance Testing Type Comparison

	Primary Goal	Key Metrics	Scenario
Load Testing	Verify stability under <i>normal</i> expected load.	Response Time, Throughput, Error Rate.	Peak hour user traffic.
Stress Testing	Identify system <i>bottlenecks</i> and breaking points.	CPU/RAM limits, HTTP 500 errors.	Load exceeding design capacity.
Spike Testing	Test system <i>resilience</i> .	Recovery capability, avoiding full crashes.	Flash Sales, sudden traffic surges.

Performance Testing Type Comparison

	Primary Goal	Key Metrics	Scenario
Volume Testing	Evaluate performance with <i>large datasets</i> .	Database retrieval speed (DB I/O).	Massive backend data growth.
Soak Testing	Assess <i>continuous</i> load handling.	Connection management, proper open/close states.	Sustained moderate load without interruption.
Endurance Testing	Test durability over <i>extended periods</i> .	Memory leaks, resource exhaustion.	Running the system continuously for hours/days.

Why: The Business & Technical Imperative



Simulate disaster.

Find the structural breaking point before it happens in production.

> 3 seconds load

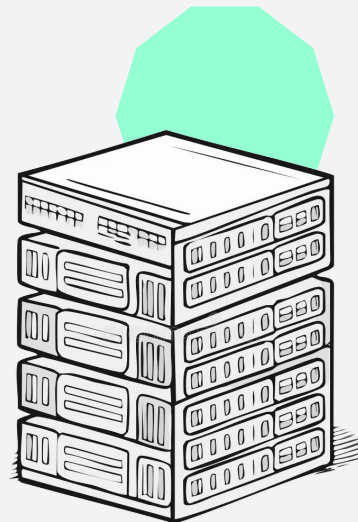
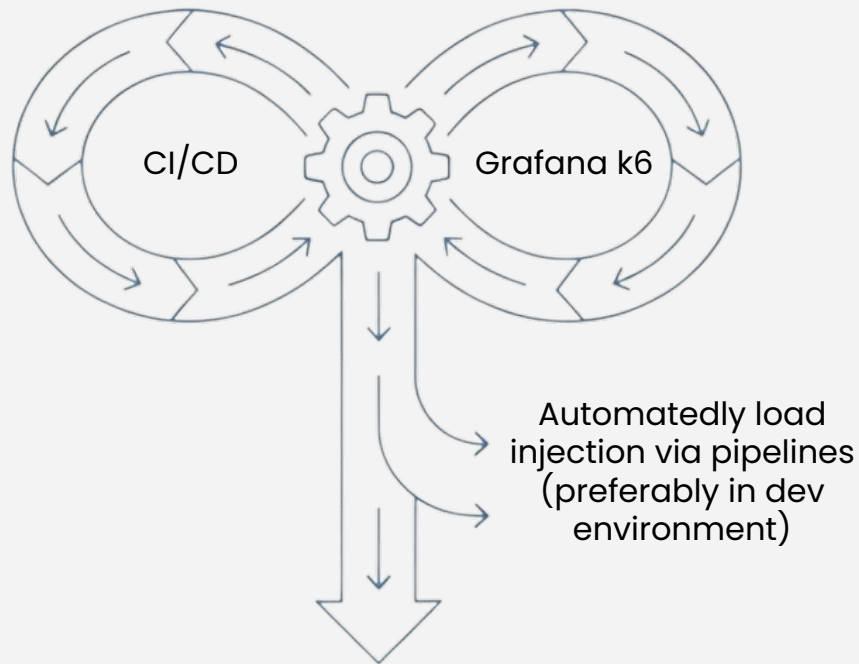
time = 53%

Drop-off (*)

Performance is not just technical.

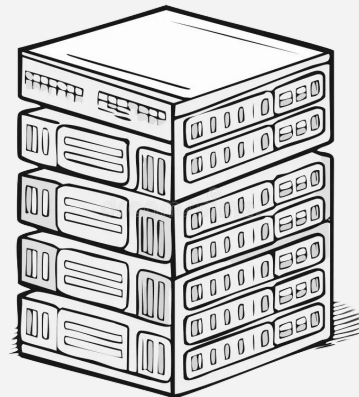
Performance is revenue.

Where: Infrastructure & Execution



Test Environment

=



Production

Testing on weak servers or empty databases yields unreliable test result.

When: Test timing & thresholds

Shift-Left Testing

Test early, test often.
Trigger tests upon major feature merges.

When to stop?

When metrics meet defined thresholds.

Who: Human in the loop

Product Owner

Defined KPIs based on expected user behavior.

Dev / QA

Empowered to write scripts and inject load using modern tooling.

The end-user

The ultimate beneficiaries of a rigorous tested, crash-free system.



02

— The Testing Process

How?

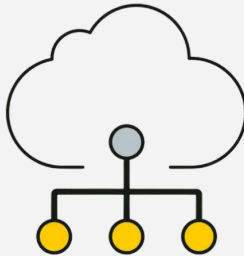
The Testing Process

1. Identify test environment

3. Plan and design

2. Determine performance criteria

4. Configure test environment

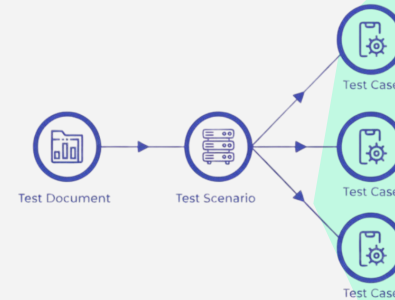


Identify test environment

<2s

Response Time

Determine performance criteria



Plan and design

The Testing Process

5. Implement
test design

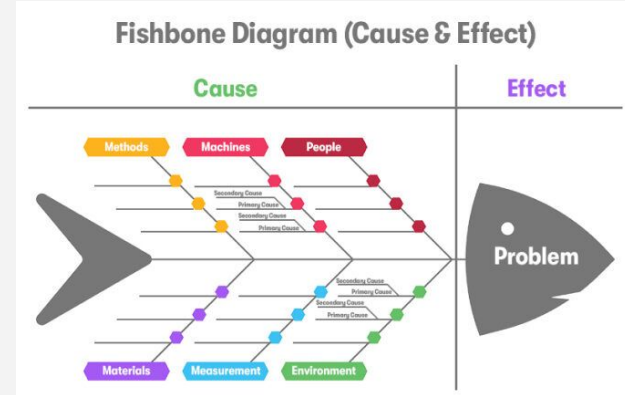
7. Analyze and
retest

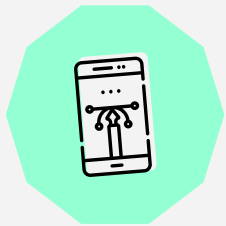
6. Run test

```
// Bắt token từ Response của API /login
Field: Response Body
Regex: "access_token": "(.*)"
Template: $1$
Match No: 1
Variable Name: AUTH_TOKEN
```

Implement test design

Analyze and retest





03

— Testing Scenarios

Traditional &
AI-Augmented

EShop-SUT

E-COMMERCE PLATFORM PERFORMANCE TESTING



Goal: Ensure a smooth shopping experience, prevent cart abandonment, and handle high traffic.



STRATEGIES

- **Load Testing**
Simulate peak-hour traffic to verify system stability.
- **Stress Testing**
Push beyond normal limits to find the bottlenecks.
- **Spike Testing**
Evaluate system resilience during sudden traffic surges.

KEY METRICS

- **Response Time < 2s**
- **Throughput > 100 checkouts/min**
- **Error Rate < 1%**
- **Resource Usage**

Main Scenario

Simulating peak daily traffic (20-50 concurrent users) for 10 minutes to verify SLA compliance.



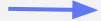
Step 1. Login

SLA: $\leq 700\text{ms}$



Step 2. Load 500 Products

SLA: $\leq 800\text{ms}$



Step 3. Search Products

SLA: $\leq 1\text{s}$



Step 4. Add to Cart

SLA: $\leq 700\text{ms}$



Step 5. Apply Coupon

SLA: $\leq 800\text{ms}$



Step 6. Checkout Submit

SLA: $\leq 1.5\text{s}$



Stress Testing

Gradually increase the load from 50 to 100+ concurrent users executing the entire flow.



Volume Testing

Evaluate system performance when interacting with an exponentially larger dataset in the backend.



Spike Testing

Simulate a sudden, massive surge of users simultaneously attempting to buy a limited-stock item.

Traditional Tool - k6



Core Functions

- Simulate concurrent users
- Execute scripted test flows
- Validate checks & thresholds
- Export metrics and reports

Best fit: QA/Dev team already knows the system flow and can maintain test scripts manually.

Workspace

```
k6-project/  
|-- scripts/  
|   |-- login_test.js  
|-- config/  
|   |-- options.js  
|-- data/  
|   |-- users.csv  
|-- results/  
|-- reports/
```

Workflow



Script



CLI Run



Load



Metrics



Manual Read

Traditional Tool - *k6*



Input

- Base URL
- VUs
- Duration / Stages
- Test Data
- Thresholds

Output

```
Test Status: PASSED
Virtual Users: 50
Total Requests: 15,240

Average Response Time: 245 ms
P95 Response Time: 480 ms
Throughput: 50.8 req/s
Error Rate: 0.8%
Checks Passed: 99.2%

Thresholds:
p95 < 500 ms      ✓ PASSED
error rate < 2%   ✓ PASSED
```

Interpretation: k6 gives reliable raw metrics, but bottleneck analysis is mostly manual.

AI-Augmented Tool - *Grafana k6 AI / k6 Studio*



Core Functions

- Generate tests from prompts or recordings
- Parameterize data and dynamic values
- Validate requests and scripts visually
- Explain results and suggest fixes

Best fit: team wants faster test creation, less manual scripting, and easier result interpretation.

Workspace

```
ai-k6-project/  
|-- recordings/  
|   |-- login_flow.har  
|-- scripts/  
|   |-- generated_test.js  
|-- test-data/  
|-- results/  
|-- dashboards/
```

Workflow



Intent



Recording



Script



Validate



AI Readout

AI-Augmented Tool - *Grafana k6 AI / k6 Studio*



Input

- Test Goal
- Target URL
- User Flow
- Workload
- SLA / SLO
- AI Context

Output

```
Test Status: WARNING
Virtual Users: 100
Total Requests: 28,500

Average Response Time: 420 ms
P95 Response Time: 1.24 s
Throughput: 78.3 req/s
Error Rate: 3.4%

Detected Bottleneck:
POST /api/login
P95: 1.82 s
Error Rate: 6.1%
```

AI Analysis

The system remained stable under 100 users, but login exceeded the 800 ms SLA.

Possible cause:

Authentication database queries slowed down as traffic increased.

Recommendation:

Add indexes, cache repeated lookups, and retest login under the same workload.

Difference from traditional output: the AI layer turns metrics into diagnosis + next action.

AI-Augmented Tool - *Grafana k6 AI / k6 Studio*



Operating Principles

- AI as a Co-Pilot
- Intent-Driven Execution
- Automated Diagnostics

Core Functions

- Prompt-to-Test
- Smart Correlation
- Root-Cause Analysis

Output



AI Performance Insight:

1. Test Summary (Natural Language): The load test successfully completed with 50 Virtual Users over 5 minutes. The overall error rate is low (1.5%), but performance degraded significantly under peak load.

- Natural Language Reports
- Automated Diagnostics
- Actionable Recommendations

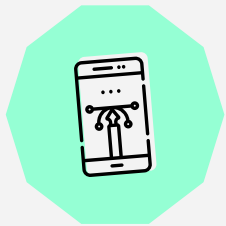




04

—
DEMO
—

Run the tests!



05

— Class Activities

Run the tests!



06

— Conclusion & QnA

The End